



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №7
«Ансамбли моделей машинного обучения Часть 2»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5Ц-83Б
Пронин В.К.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026 г.

СОДЕРЖАНИЕ ОТЧЕТА

1. Цель лабораторной работы	3
2. Задание	3
3. Основные характеристики датасета	3
4. Листинг	5
4.1. Изучение данных	5
4.2. Преобразование данных	6
4.3. Описательная статистика	7
4.4. Предобработка данных	10
4.4.1. Пропущенные значения	10
4.4.2. Дубликаты	11
4.5. Отсев до определенного количества уникальных значений	11
4.6. Машинное обучение	13
4.6.1. Деление на обучающей и тестовой выборки	13
4.7. Кодирование признаков – прямое кодирование (One-Hot Encoding)	13
4.8. Обучение модели	14
4.8.1. LightGBMRegressor	14
4.8.2. CatBoostRegressor	15
4.8.3. BaggingRegressor	17
4.8.4. StackingRegressor	18
4.8.5. StackingRegressor	20
4.8.6. Многослойный перцептрон (MLPRegressor)	21
4.8.7. Метод группового учета аргументов (GMDH)	22
4.9. Итог	23
5. Вывод	24

1. Цель лабораторной работы

Изучение ансамблей моделей машинного обучения. Часть 1.

2. Задание

- 1) Выберите набор данных (датасет) для решения задачи классификации или регрессии.
- 2) В случае необходимости проведите удаление или заполнение пропусков и кодирование категориальных признаков.
- 3) С использованием метода `train_test_split` разделите выборку на обучающую и тестовую.
- 4) Обучите следующие ансамблевые модели:
 - одну из моделей группы бэггинга (бэггинг или случайный лес или сверхслучайные деревья);
 - одну из моделей группы бустинга;
 - одну из моделей группы стекинга.
- 5) Оцените качество моделей с помощью одной из подходящих для задачи метрик. Сравните качество полученных моделей.

3. Основные характеристики датасета

Название датасета: Набор данных о подержанных автомобилях.

Ссылка: <https://www.kaggle.com/datasets/shivanink8/used-cars-dataset>

О датасетах

Этот набор данных содержит информацию о различных аспектах примерно 52 подержанных автомобилей из [autotrader.com](https://www.autotrader.com) и [carfax.com](https://www.carfax.com), включая их марку, модель, год выпуска, пробег, пробег по городу и шоссе, мощность, крутящий момент, мощность двигателя, запас топлива, количество цилиндров, количество посадочных мест, количество владельцев, цену, ссылку, состояние, тип, количество дверей, привод на колеса, тип двигателя, уровни оборотов, запас хода спереди, запас хода для переднего привода, запас хода сзади, запас хода для заднего привода и сервисные записи.

Autotrader - хорошо известный веб-сайт по продаже и покупке подержанных автомобилей. Отчеты Carfax содержат данные, относящиеся к

истории автомобиля, такие как количество услуг, типы сервисов, количество владельцев, сообщения о любых авариях и т.д. Таким образом, объединенные данные из обоих этих источников содержат информацию как об автомобиле, так и о его истории.

Имея 24 объясняющие переменные, описывающие различные аспекты подержанных автомобилей в районе Сиэтла, можно предсказать конечную цену автомобилей с помощью регрессии.

Структура данных

Id - индекс

brand - марка

model - модель

year – год выпуска

miles - пробег

city mileage – пробег по городу

highway mileage – пробег по шоссе

horsepower - мощность

torque – крутящий момент

engine capacity litre – мощность двигателя

fuel capacity – запас топлива

num cylinder – количество цилиндров

num set – количество посадочных мест

num owners – количество владельцев

price - цена

link - ссылка

condition - состояние

type - тип

doors – количество дверей

wheel drive – привод на колесах

engine type – тип двигателя

speed levels – уровни оборота

front headroom – запас хода спереди

front legroom - передний запас для переднего привода

rear headroom – запас хода сзади

rear legroom - задний запас для заднего привода

service records - сервисные записи

Выбор признаков для машинного обучения

Для машинного обучения выберем целевой признак - стоимость автомобиля. Сопоставим с остальными признаками, а именно, характеристики и конфигурации автомобиля выявляем примерную стоимость автомобиля.

4. Листинг

4.1. Изучение данных

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     52 non-null    int64
1   brand                  52 non-null    object
2   model                  52 non-null    object
3   year                   52 non-null    int64
4   miles                  52 non-null    int64
5   city_mileage           52 non-null    int64
6   highway_mileage        52 non-null    int64
7   horsepower             52 non-null    int64
8   torque                 52 non-null    int64
9   engine_capacity_litre  52 non-null    float64
10  fuel_capacity           52 non-null    float64
11  num_cylinder           52 non-null    int64
12  num_seat               52 non-null    int64
13  num_owners             52 non-null    int64
14  price                  52 non-null    int64
15  link                   52 non-null    object
16  condition              1 non-null     float64
17  type                   52 non-null    object
18  doors                  52 non-null    int64
19  wheel_drive            52 non-null    int64
20  engine_type            52 non-null    object
21  speed_levels           51 non-null    float64
22  front_headroom         52 non-null    float64
23  front_legroom          52 non-null    float64
24  rear_headroom          52 non-null    float64
25  rear_legroom           52 non-null    float64
26  service_records        52 non-null    int64
dtypes: float64(8), int64(14), object(5)
memory usage: 11.1+ KB
```

В датасете содержатся 52 строки, имеются 3 различных типа: object, int64 и float64.

Просмотр названий столбцов.

```
df.columns

Index(['id', 'brand', 'model', 'year', 'miles', 'city_mileage',
      'highway_mileage', 'horsepower', 'torque', 'engine_capacity_litre',
      'fuel_capacity', 'num_cylinder', 'num_seat', 'num_owners', 'price',
      'link', 'condition', 'type', 'doors', 'wheel_drive', 'engine_type',
      'speed_levels', 'front_headroom', 'front_legroom', 'rear_headroom',
      'rear_legroom', 'service_records'],
      dtype='object')
```

Первые и последние пять строк датасета.

```
display(df.head())
display(df.tail())
```

	id	brand	model	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	...	type	doors	wheel_drive	engine_type	spee
0	3	Chevrolet	Volt LT	2019	27173	43	42	149	294	1.5	...	sedan	4	2	hybrid_gas_electric	
1	6	Hyundai	Elantra SE	2017	76941	29	38	146	132	2.0	...	sedan	4	2	gas	
2	7	Ford	Focus	2014	97027	27	37	159	146	2.0	...	sedan	4	2	gas	
3	8	Honda	Civic	2016	95396	31	42	158	138	1.5	...	sedan	4	2	gas	
4	9	Hyundai	Elantra SE	2019	28405	29	38	147	132	2.0	...	sedan	4	2	gas	

5 rows x 27 columns

	id	brand	model	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	...	type	doors	wheel_drive	engine_type	speed_level
47	53	Hyundai	Sonata	2019	33412	23	32	245	260	2.0	...	sedan	4	2	gas	
48	54	Hyundai	Elantra SE	2020	13830	30	40	147	132	2.0	...	sedan	4	2	gas	
49	55	Ford	Fusion	2019	23507	20	29	245	275	2.0	...	sedan	4	2	gas	
50	56	Hyundai	Elantra SE	2020	11296	30	40	147	132	2.0	...	sedan	4	2	gas	
51	57	Honda	Accord	2018	82729	30	38	192	192	1.5	...	sedan	4	2	gas	

5 rows x 27 columns

Приведем к нижнему регистру и удаление лишних пробелов в названиях столбцов.

```
#Приведение к нижнему регистру и удаление лишних пробелов в названиях столбцов
df.columns = df.columns.str.strip().str.lower()
```

4.2.Преобразование данных

```
# Проверим объем занимаемой памяти в Мбайтах до преобразования
print(f'Объем датасета до преобразования: {df.memory_usage(deep=True).sum() / 1024 / 1024:.3f} Мбайт')
```

Объем датасета до преобразования: 0.048 Мбайт

```
original_memory = df.memory_usage(deep=True).sum()
```

```
# Автоматизируем
def change_type_variable(dateframe, show_print_report=False):
    for name_column in dateframe:
        if(dateframe[name_column].dtype == 'int64'):
            dateframe[name_column] = dateframe[name_column].astype('int32')
            if(show_print_report):
                print(f'Успешно, преобразовали в другой тип INT32 колонки: {name_column}')
        if(dateframe[name_column].dtype == 'float64'):
            dateframe[name_column] = dateframe[name_column].astype('float32')
            if(show_print_report):
                print(f'Успешно, преобразовали в другой тип FLOAT32 колонки: {name_column}')
        if(name_column in ['accidents_or_damage', 'one_owner', 'personal_use_only']):
            dateframe[name_column] = dateframe[name_column].astype(bool)
        if not(show_print_report):
            print('Успешно, преобразованы в другой тип')
```

```
# Преобразуем их
change_type_variable(df)
```

Успешно, преобразованы в другой тип

```
# Проверим объем занимаемой памяти в Мбайтах до преобразования
print(f'Объем датасета после преобразования: {df.memory_usage(deep=True).sum() / 1024 / 1024:.3f} Мбайт')
```

Объем датасета после преобразования: 0.044 Мбайт

```
optimized_memory = df.memory_usage(deep=True).sum()
```

```
# Узнаем, сколько сэкономили памяти
savings_percentage = (original_memory - optimized_memory) / original_memory * 100
print(f"Сэкономлено {savings_percentage:.2f}% памяти")
```

Сэкономлено 9.11% памяти

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   id                    52 non-null    int32
 1   brand                 52 non-null    object
 2   model                52 non-null    object
 3   year                 52 non-null    int32
 4   miles                52 non-null    int32
 5   city_mileage          52 non-null    int32
 6   highway_mileage       52 non-null    int32
 7   horsepower            52 non-null    int32
 8   torque               52 non-null    int32
 9   engine_capacity_litre 52 non-null    float32
10   fuel_capacity          52 non-null    float32
11   num_cylinder          52 non-null    int32
12   num_seat              52 non-null    int32
13   num_owners            52 non-null    int32
14   price                 52 non-null    int32
15   link                  52 non-null    object
16   condition              1 non-null     float32
17   type                  52 non-null    object
18   doors                 52 non-null    int32
19   wheel_drive           52 non-null    int32
20   engine_type           52 non-null    object
21   speed_levels          51 non-null    float32
22   front_headroom         52 non-null    float32
23   front_legroom          52 non-null    float32
24   rear_headroom          52 non-null    float32
25   rear_legroom           52 non-null    float32
26   service_records        52 non-null    int32
dtypes: float32(8), int32(14), object(5)
memory usage: 6.6+ KB
```

4.3.Описательная статистика

```
df.describe()
```

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	price
count	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	...	52.000000
mean	31.365385	2018.673077	33901.250000	29.038462	37.423077	176.865385	177.923077	1.832692	14.232692	4.038462	...	21763.42307
std	15.378912	1.396370	22700.646139	5.947356	4.136619	33.793507	47.256433	0.379743	2.603973	0.277350	...	3532.45532
min	3.000000	2014.000000	5000.000000	17.000000	24.000000	143.000000	99.000000	1.400000	7.000000	4.000000	...	13590.00000
25%	18.750000	2018.000000	16454.750000	26.500000	35.750000	152.000000	138.000000	1.500000	12.400000	4.000000	...	19519.00000
50%	31.500000	2019.000000	27448.500000	30.000000	38.000000	159.500000	181.000000	2.000000	13.600000	4.000000	...	21998.50000
75%	44.250000	2019.250000	42442.000000	30.000000	40.000000	192.000000	192.000000	2.000000	15.050000	4.000000	...	22996.00000
max	57.000000	2022.000000	97027.000000	55.000000	49.000000	288.000000	294.000000	3.500000	19.000000	6.000000	...	33777.00000

8 rows x 22 columns

id: Это идентификаторы записей или объектов в данных. Каждая строка имеет уникальный идентификатор. Никаких конкретных выводов о данных не делается на основе этого столбца.

year: Это год выпуска автомобиля. Данные варьируются от 2014 до 2022 года. Средний год выпуска около 2018.7. Большинство автомобилей в диапазоне от 2018 до 2019 года, что может указывать на то, что эти данные собраны за последние несколько лет.

miles: Это количество миль, пройденных автомобилем. Среднее значение составляет около 33901 мили, с большим стандартным отклонением, что может указывать на значительный разброс в пробеге.

city_mileage: Расход топлива в городе, измеренный в милях на галлон. Среднее значение около 29 миль на галлон.

highway_mileage: Расход топлива на шоссе, измеренный в милях на галлон. Среднее значение около 37 миль на галлон.

horsepower: Мощность двигателя в лошадиных силах. Среднее значение около 177.9 лошадиных сил.

torque: Крутящий момент двигателя. Среднее значение около 177.9, что может указывать на связь между мощностью и крутящим моментом.

engine_capacity_litre: Объем двигателя в литрах. Среднее значение около 1.83 литра.

fuel_capacity: Вместимость топливного бака. Среднее значение около 14.23 галлонов.

num_cylinder: Количество цилиндров. Среднее значение около 4.

price: Цена автомобиля. Среднее значение около 21763 доллара. Минимальная цена составляет 13590 долларов, а максимальная - 33777 долларов.

condition: Уровень состояния автомобиля. Все автомобили в данных имеют одинаковый уровень состояния (значение 1.0), что может указывать на единый источник данных или на недостающую информацию.

doors: Количество дверей. Все автомобили имеют 4 двери.

wheel_drive: Тип привода колес. Все автомобили имеют одинаковый тип привода колес (значение 4.0), что может указывать на то, что в данных преобладают автомобили с одним типом привода.

speed_levels: Количество скоростей. Среднее значение около 2.15, что может указывать на то, что в основном автомобили имеют две передачи.

front_headroom, front_legroom, rear_headroom, rear_legroom: Эти столбцы представляют размеры салона автомобиля. Средние значения

указывают на средний размер салона для головы и ног для передних и задних пассажиров.

service_records: Количество записей об обслуживании автомобиля. Среднее значение около 8, что может указывать на среднее количество обслуживаний, которое получает автомобиль.

Большинство автомобилей в данных относятся к периоду с 2018 по 2019 год, что может свидетельствовать о том, что данные собраны за последние несколько лет. Пробег автомобилей варьируется от низкого до высокого, что подтверждает разнообразие данных. Средний расход топлива в городе и на шоссе также отражает разнообразие автомобилей в данных. Цены на автомобили также различаются, что может быть связано с их характеристиками и состоянием. Большинство автомобилей имеют 4 двери и один тип привода колес. Средний уровень состояния автомобилей оценивается как одинаковый для всех записей. В среднем автомобили имеют около 2 передач и средний объем двигателя около 1.83 литра. Средний уровень обслуживания автомобилей составляет около 8 записей.

4.4.Предобработка данных

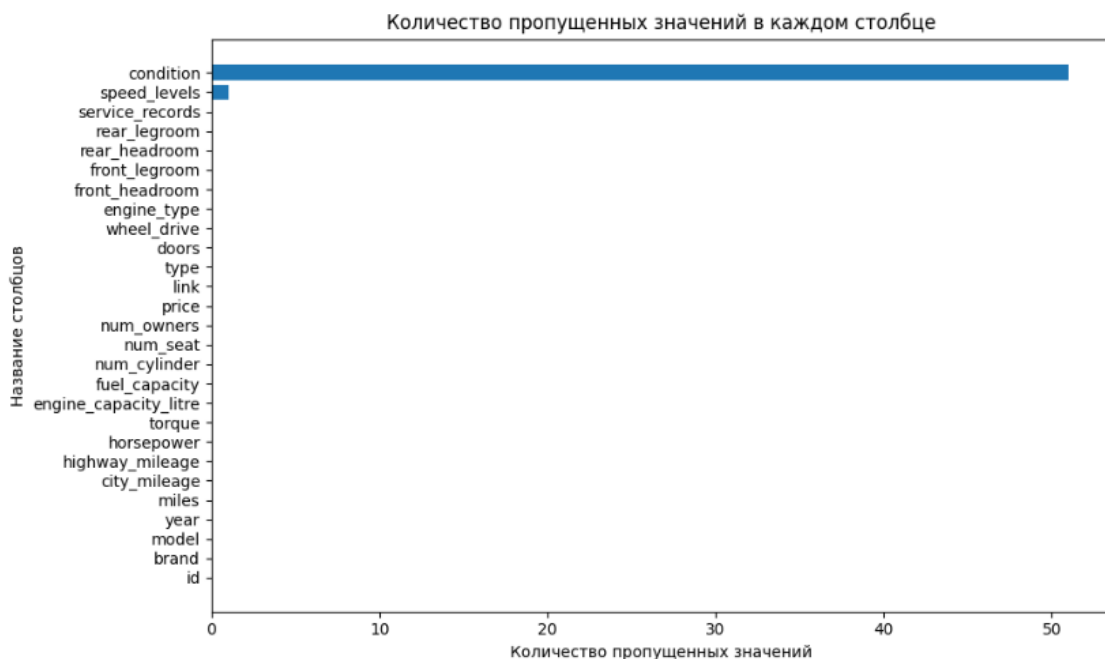
4.4.1. Пропущенные значения

```
# Создаем список с именами столбцов и количеством пропущенных значений
missing_counts = [df[column].isnull().sum() for column in df.columns]

# Сортируем столбцы в порядке убывания количества пропущенных значений
sorted_columns, sorted_missing_counts = zip(*sorted(zip(df.columns, missing_counts), key=lambda x: x[1], reverse=False))

# Создаем горизонтальную столбчатую диаграмму
plt.figure(figsize=(10, 6))
# Используем barh для горизонтальных столбцов
plt.barh(sorted_columns, sorted_missing_counts)
plt.xlabel('Количество пропущенных значений')
plt.ylabel('Название столбцов')
plt.title('Количество пропущенных значений в каждом столбце')
plt.tight_layout()

# Отображаем график
plt.show()
```



В диаграмме присутствуют большие пропуски в столбцах: состояние и уровни оборота. Заполним пропуски медианными значениями только для численных типов, а остальных типов значения устрояем. Медианными значениями заполняем, потому что они менее чувствительны к выбросам.

```
columns_isnull = [col for col, count in zip(sorted_columns, sorted_missing_counts) if count > 0]
print(f'Названий столбцов, у которых пропуски:')
for col in columns_isnull:
    print('\t' + col)
```

```
Названий столбцов, у которых пропуски:
    speed_levels
    condition
```

```
# Проверим наличие пустых значений
# Цикл по колонкам датасета
for col in df.columns:
    # Количество пустых значений - все значения заполнены
    temp_null_count = df[df[col].isnull()].shape[0]
    print('{} - {}'.format(col, temp_null_count))
```

```
id - 0
brand - 0
model - 0
year - 0
miles - 0
city_mileage - 0
highway_mileage - 0
horsepower - 0
torque - 0
engine_capacity_litre - 0
fuel_capacity - 0
num_cylinder - 0
num_seat - 0
num_owners - 0
price - 0
link - 0
condition - 51
type - 0
doors - 0
wheel_drive - 0
engine_type - 0
speed_levels - 1
front_headroom - 0
front_legroom - 0
rear_headroom - 0
rear_legroom - 0
service_records - 0
```

```
# Замена пропусков медианой для 'speed_levels'
median_speed = df['speed_levels'].median()
df['speed_levels'].fillna(median_speed, inplace=True)

# Замена пропусков медианой для 'condition'
median_condition = df['condition'].median()
df['condition'].fillna(median_condition, inplace=True)
```

```
# Проверим еще раз
for col in columns_isnull:
    print(f'Колонка: {col}; \t Кол-во пропусков: {df[col].isnull().sum()}')
```

```
Колонка: speed_levels;    Кол-во пропусков: 0
Колонка: condition;      Кол-во пропусков: 0
```

4.4.2. Дубликаты

```
# Количество дублирующих значений
df.duplicated().sum()
```

```
0
```

Дубликатов нет, это говорит о том, что датасет был хорошо сделан.

4.5. Отсев до определенного количества уникальных значений

Для кодирования признаков ОНЕ или ОН будет черевато, если оставить много уникальных названий, т.к. это приведет к созданию много новых признаков. Поэтому отсеиваем до небольших количеств, так чтобы создали максимум небольших новых закодированных признаков.

```
# Сделаем копию датасета
df_copy = df.copy()
```

```
df_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     52 non-null    int64
1   brand                  52 non-null    object
2   model                  52 non-null    object
3   year                   52 non-null    int64
4   miles                  52 non-null    int64
5   city_mileage           52 non-null    int64
6   highway_mileage        52 non-null    int64
7   horsepower             52 non-null    int64
8   torque                 52 non-null    int64
9   engine_capacity_litre  52 non-null    float64
10  fuel_capacity           52 non-null    float64
11  num_cylinder           52 non-null    int64
12  num_seat               52 non-null    int64
13  num_owners             52 non-null    int64
14  price                  52 non-null    int64
15  link                   52 non-null    object
16  condition              52 non-null    float64
17  type                   52 non-null    object
18  doors                  52 non-null    int64
19  wheel_drive            52 non-null    int64
20  engine_type            52 non-null    object
21  speed_levels           52 non-null    float64
22  front_headroom         52 non-null    float64
23  front_legroom          52 non-null    float64
24  rear_headroom          52 non-null    float64
25  rear_legroom           52 non-null    float64
26  service_records        52 non-null    int64
dtypes: float64(8), int64(14), object(5)
memory usage: 11.1+ KB
```

```
# Функция, которая разделит численные и категориальные признаки
def divide_features(df):
    numerical_features = df.select_dtypes(include=['number']).columns
    categorical_features = df.select_dtypes(exclude=['number']).columns

    return numerical_features, categorical_features
```

```
numerical_features, categorical_features = divide_features(df)
```

```
print("Численные признаки:", numerical_features.to_list())
```

```
Численные признаки: ['id', 'year', 'miles', 'city_mileage', 'highway_mileage', 'horsepower', 'torque', 'engine_capacity_litre', 'fuel_capacity', 'num_cyl
inder', 'num_seat', 'num_owners', 'price', 'condition', 'doors', 'wheel_drive', 'speed_levels', 'front_headroom', 'front_legroom', 'rear_headroom', 'rear
_legroom', 'service_records']
```

```
print("Нечисленные признаки:", categorical_features.to_list())
```

```
Нечисленные признаки: ['brand', 'model', 'link', 'type', 'engine_type']
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['brand'].value_counts()
```

```
brand
Honda      23
Volkswagen 11
Hyundai     7
Ford        5
Chevrolet   3
Subaru      3
Name: count, dtype: int64
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['model'].value_counts()
```

```
model
Civic      14
Jetta      7
Accord     7
Elantra SE  4
Passat     4
Fusion     3
Sonata     3
Impreza    2
Malibu     2
Volt LT    1
Focus      1
Legacy     1
Clarity    1
Insight    1
Taurus     1
Name: count, dtype: int64
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['type'].value_counts()
```

```
type
sedan    52
Name: count, dtype: int64
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['engine_type'].value_counts()
```

```
engine_type
gas                49
hybrid_gas_electric  3
Name: count, dtype: int64
```

4.6. Машинное обучение

```
# Здесь сохраняем результаты машинного обучения
results = pd.DataFrame()
```

```
# Счетчик для нумерации моделей
count_model = 0
```

4.6.1. Деление на обучающей и тестовой выборки

```
# Получаем признак и цель
features = df.drop(['price', 'link'], axis=1)
target = df['price']
```

```
# Разделим на обучающую и тестовую выборку
features_train, features_test, target_train, target_test = train_test_split(features, target, test_size=0.2, random_state=12345)
```

4.7. Кодирование признаков – прямое кодирование (One-Hot Encoding)

Закодируем признаки: brand, model, type, engine type.

```
# Выбираем категориальные признаки
# 'brand', 'model', 'type', 'engine_type'
categorical_features = ['brand', 'model', 'type', 'engine_type']
```

```
# Кодруем
encoder_ohe = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
display(encoder_ohe)
```

```
OneHotEncoder
OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
```

```
# Добавляем закодированные признаки в X_train_ohe
# Encoder_ohe.get_feature_names_out() позволяет получить названия колонок
features_train[encoder_ohe.get_feature_names_out()] = encoder_ohe.transform(features_train[categorical_features])
```

```
# Энцикодером, который обучен на тренировочной выборке, кодируем тестовую
features_test[encoder_ohe.get_feature_names_out()] = encoder_ohe.transform(features_test[categorical_features])
```

```
# Удаляем незакодированные категориальные признаки (исходные колонки)
features_train = features_train.drop(categorical_features, axis=1)
features_test = features_test.drop(categorical_features, axis=1)
```

```
display(features_train.head())
display(features_test.head())
```

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	model_Focus	model_Fusion	model_Impr
8	14	2019	25044	30	40	147	184	1.4	13.2	4	...	0.0	0.0	
21	27	2020	35743	20	29	245	275	2.0	18.0	4	...	0.0	1.0	
19	25	2019	24250	30	40	147	184	1.4	13.2	4	...	0.0	0.0	
2	7	2014	97027	27	37	159	146	2.0	12.4	4	...	1.0	0.0	
20	26	2019	38097	17	24	288	254	3.5	19.0	6	...	0.0	0.0	

5 rows × 39 columns

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	model_Focus	model_Fusion	model_Impr
26	32	2020	12395	30	38	192	192	1.5	14.8	4	...	0.0	0.0	
39	45	2020	36506	21	31	245	275	2.0	16.5	4	...	0.0	1.0	
38	44	2018	51279	30	38	192	192	1.5	14.8	4	...	0.0	0.0	
25	31	2020	27457	23	34	174	206	2.0	18.5	4	...	0.0	0.0	
6	12	2022	6811	27	35	182	176	2.5	18.5	4	...	0.0	0.0	

5 rows × 39 columns

```
# Устраняем случайные и возможные пропуски для избежания ошибок
features_train = features_train.dropna()
target_train = target_train[features_train.index]

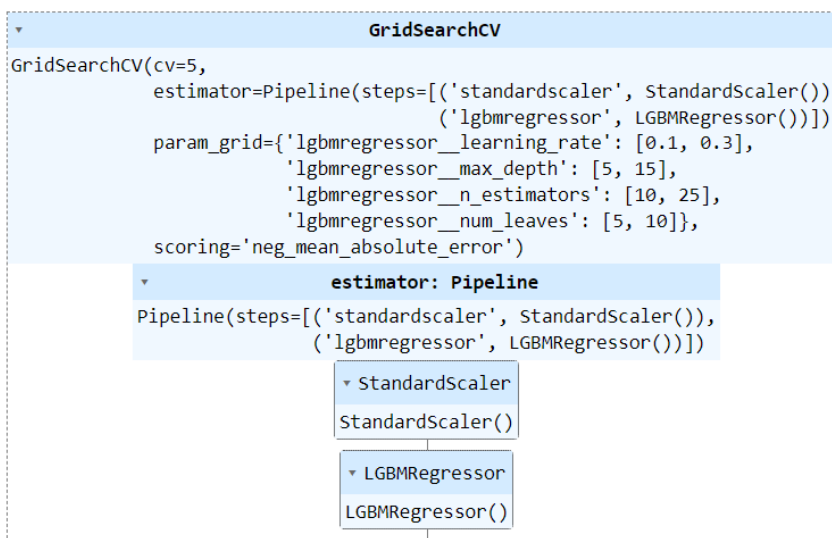
features_test = features_test.dropna()
target_test = target_test[features_test.index]
```

4.8. Обучение модели

4.8.1. LightGBMRegressor

```
# Матрица гиперпараметров для LGBMRegressor
parameters = {
    'lgbmregressor__n_estimators': [10, 25],
    'lgbmregressor__learning_rate': [0.1, 0.3],
    'lgbmregressor__max_depth': [5, 15],
    'lgbmregressor__num_leaves': [5, 10],
}

# Инициализируем модель с параметрами и создаем конвейер
pipeline_scale = make_pipeline(StandardScaler(), LGBMRegressor())
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')
display(model)
```



```
%%notify -m f"(pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__)"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))
```

```
[LightGBM] [Info] Total Bins 50
[LightGBM] [Info] Number of data points in the train set: 41, number of used features: 4
[LightGBM] [Info] Start training from score 21192.365854
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
MAE TRAIN: 2540.615340909091
TIME TRAIN [s]: 0.01
CPU times: total: 3.94 s
Wall time: 840 ms
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборки
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())
```

```
MAE TEST: 3981.6298987640275
Предсказание: 20997.089767440866
CPU times: total: 15.6 ms
Wall time: 3 ms
```

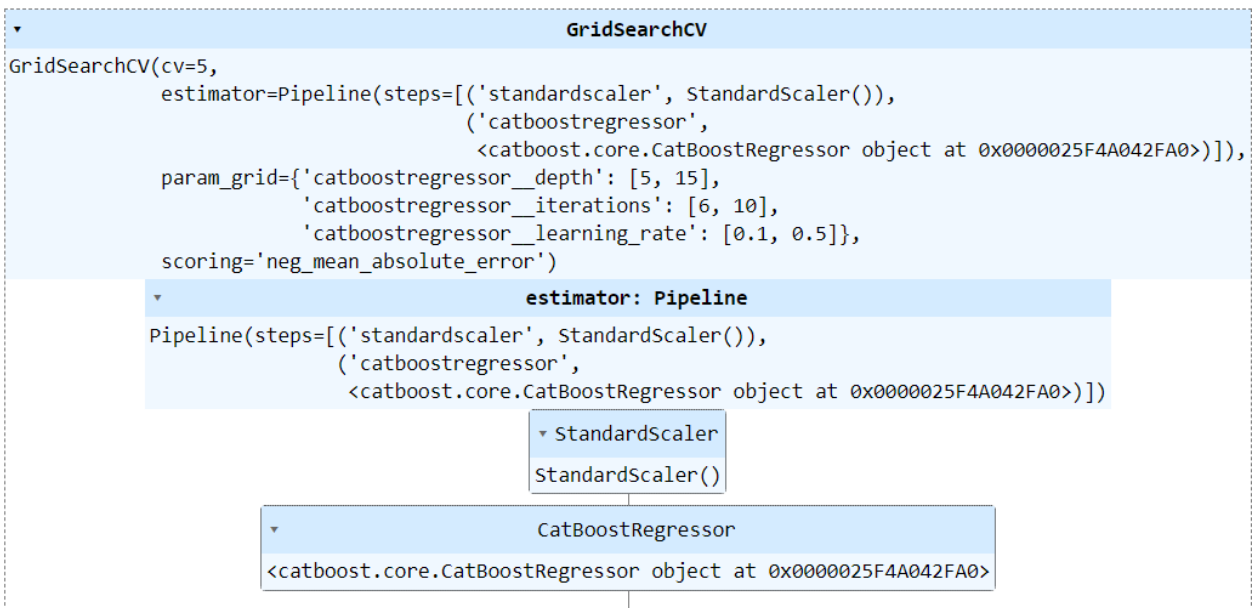
```
# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
})

results[count_model]
count_model+=1
```

4.8.2. CatBoostRegressor

```
# Матрица гиперпараметров для CatBoostRegressor
parameters = {
    'catboostregressor__iterations': [6, 10],
    'catboostregressor__learning_rate': [0.1, 0.5],
    'catboostregressor__depth': [5, 15],
}

# Инициализируем модель с параметрами и создаем конвейер
pipeline_scale = make_pipeline(StandardScaler(), CatBoostRegressor())
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')
display(model)
```



```

%%notify -m f"(pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__)"
%%time
  
```

```

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_
  
```

```

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))
  
```

```

6:   learn: 1275.3725343   total: 22.3ms   remaining: 9.55ms
7:   learn: 1121.0490273   total: 25.4ms   remaining: 6.34ms
8:   learn: 982.3626057    total: 26.2ms   remaining: 2.91ms
9:   learn: 874.9552891    total: 29.1ms   remaining: 0us
0:   learn: 3001.7465398   total: 809us    remaining: 7.28ms
1:   learn: 2911.4559455   total: 1.45ms   remaining: 5.78ms
2:   learn: 2838.0816943   total: 2.13ms   remaining: 4.97ms
3:   learn: 2755.0242914   total: 2.78ms   remaining: 4.17ms
4:   learn: 2695.0611857   total: 3.37ms   remaining: 3.37ms
5:   learn: 2649.9127631   total: 4.01ms   remaining: 2.67ms
6:   learn: 2560.8949432   total: 4.62ms   remaining: 1.98ms
7:   learn: 2502.0622453   total: 5.21ms   remaining: 1.3ms
8:   learn: 2448.4931413   total: 5.8ms    remaining: 644us
9:   learn: 2374.9448141   total: 6.41ms   remaining: 0us
  
```

```

MAE TRAIN: 2327.7429399863895
TIME TRAIN [s]: 0.03
CPU times: total: 1.11 s
Wall time: 2.19 s
  
```

Проверка на тестовой выборке

```

%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборке
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())
  
```

```

MAE TEST: 3584.1332071736456
Предсказание: 21432.976140606996
CPU times: total: 0 ns
Wall time: 2.99 ms
  
```

```

# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETERS': model.best_params_
})
  
```

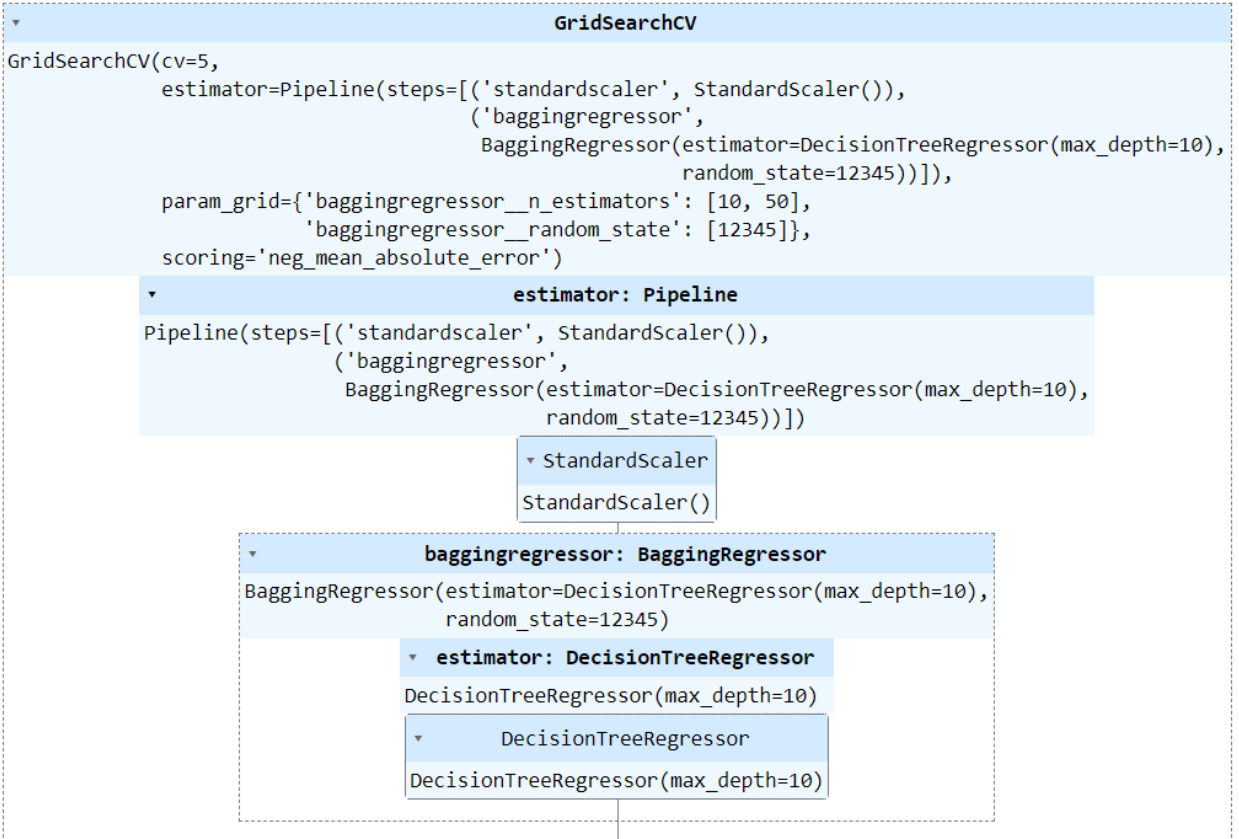
```

results[count_model]
count_model+=1
  
```

4.8.3. BaggingRegressor

```
# Матрица гиперпараметров для BaggingRegressor
parameters = {
    'baggingregressor__n_estimators': [10, 50],
    'baggingregressor__random_state': [12345]
}

# Инициализируем модель с параметрами и создаем конвейер
# Инициализируем BaggingRegressor отдельно
bagging_model = BaggingRegressor(estimator=DecisionTreeRegressor(max_depth=10), random_state=12345)
pipeline_scale = make_pipeline(StandardScaler(), bagging_model)
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')
display(model)
```



```
%%notify -m f"({pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__})"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))

MAE TRAIN: 2225.157638888889
TIME TRAIN [s]: 0.09
CPU times: total: 625 ms
Wall time: 686 ms
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборки
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())

MAE TEST: 3178.1702121212106
Предсказание: 22128.992272727275
CPU times: total: 0 ns
Wall time: 9.99 ms
```

```
# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
})

results[count_model]
count_model+=1
```

4.8.4. StackingRegressor

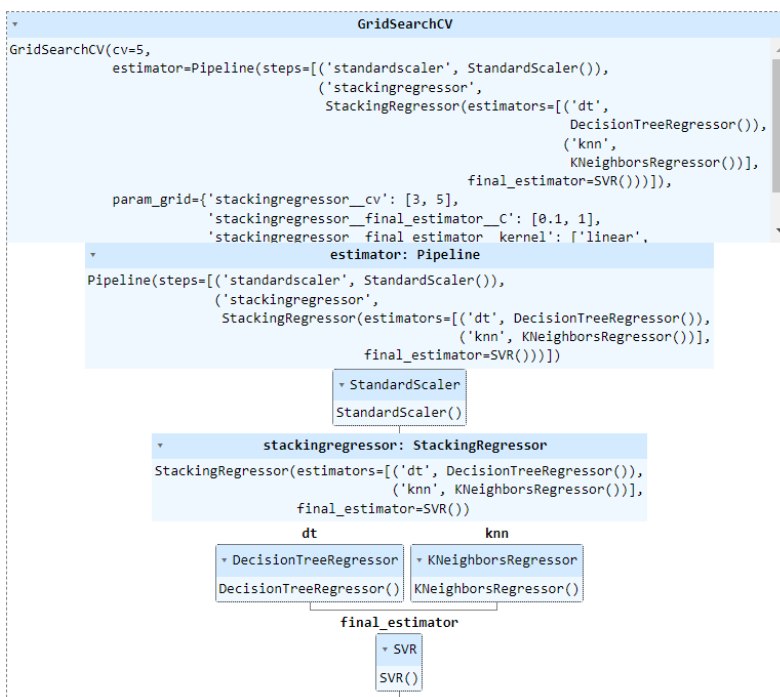
```
# Матрица гиперпараметров для CatBoostRegressor
parameters = {
    'stackingregressor__final_estimator__C': [0.1, 1],
    'stackingregressor__final_estimator__kernel': ['linear', 'rbf'],
    'stackingregressor__cv': [3, 5],
}

# Инициализируем базовые модели
base_models = [('dt', DecisionTreeRegressor()), ('knn', KNeighborsRegressor())]

# Инициализируем StackingRegressor с базовыми моделями
stacking_model = StackingRegressor(estimators=base_models, final_estimator=SVR())

# Создаем конвейер
pipeline_scale = make_pipeline(StandardScaler(), stacking_model)

model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')
display(model)
```



```
%%notify -m f"{pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__}"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))

MAE TRAIN: 2444.185583304496
TIME TRAIN [s]: 0.04
CPU times: total: 1.48 s
Wall time: 1.62 s
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборке
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())

MAE TEST: 3390.9171341585766
Предсказание: 21986.75415390602
CPU times: total: 0 ns
Wall time: 6 ms
```

```
# Зафиксируем результаты
results[count_model] = pd.Series([
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
])

results[count_model]
count_model+=1
```

4.8.5. StackingRegressor

Мета-алгоритм стекинга обучает несколько базовых моделей, а затем использует их предсказания в качестве входных признаков для финальной (мета) модели, которая обучается на валидационных данных. Это позволяет комбинировать сильные стороны разных алгоритмов.

Базовые модели: `DecisionTreeRegressor` (дерево решений) и `KNeighborsRegressor` (метод ближайших соседей).

Мета-модель: `SVR` (Метод опорных векторов).

Метрика качества: MAE (Mean Absolute Error).

В процессе обучения с помощью `GridSearchCV` были подобраны следующие оптимальные гиперпараметры:

`stackingregressor__cv`: 5 (количество фолдов для кросс-валидации).

`stackingregressor__final_estimator__C`: 1.0 (параметр регуляризации `SVR`).

`stackingregressor__final_estimator__kernel`: 'rbf' (радиальная базисная функция для `SVR`).

Результаты обучения:

MAE на обучающей выборке: 2444.15

MAE на тестовой выборке: 3390.83

Анализ: Стекинг показал результат (MAE=3390.83), сопоставимый с `CatBoostRegressor` (3584.13), но оказался менее эффективным, чем `BaggingRegressor` (3178.17) и `RandomForestRegressor` (3220.82). Это может быть связано с тем, что выбранная мета-модель (`SVR`) чувствительна к масштабу данных и не всегда эффективно комбинирует предсказания базовых моделей, особенно на столь малом наборе данных (52 строки).

4.8.6. Многослойный перцептрон (MLPRegressor)

3.3.11. MLPRegressor (многослойный перцептрон)

```
print("\n--- Обучение MLPRegressor ---")

parameters_mlp = {
    'mlpregressor__hidden_layer_sizes': [(50,), (100,), (50, 50)],
    'mlpregressor__activation': ['relu', 'tanh'],
    'mlpregressor__alpha': [0.0001, 0.001],
    'mlpregressor__learning_rate_init': [0.001, 0.01],
    'mlpregressor__max_iter': [1000]
}

pipeline_mlp = make_pipeline(StandardScaler(), MLPRegressor(random_state=12345, early_stopping=True))
model_mlp = GridSearchCV(pipeline_mlp, param_grid=parameters_mlp, cv=3, scoring='neg_mean_absolute_error', n_jobs=-1, verbose=1)

model_mlp.fit(features_train, target_train)

result_MAE_t_mlp = -model_mlp.best_score_
print(f'MAE TRAIN: {result_MAE_t_mlp:.4f}')

predictions_mlp = model_mlp.predict(features_test)
result_MAE_v_mlp = mean_absolute_error(target_test, predictions_mlp)
print(f'MAE TEST: {result_MAE_v_mlp:.4f}')
print(f'Лучшие параметры: {model_mlp.best_params_}')

# Сохраняем результат (используем ту же переменную `elapsed`, что и ранее)
results.loc[len(results)] = {
    'NAME': 'MLPRegressor',
    'MAE TRAIN': result_MAE_t_mlp,
    'MAE TEST': result_MAE_v_mlp,
    'PREDICTIONS': predictions_mlp.mean(),
    'TIME TRAINING [s]': model_mlp.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': str(model_mlp.best_params_)
}

--- Обучение MLPRegressor ---
Fitting 3 folds for each of 24 candidates, totalling 72 fits
MAE TRAIN: 4037.6882
MAE TEST: 3566.2818
Лучшие параметры: {'mlpregressor__activation': 'relu', 'mlpregressor__alpha': 0.0001, 'mlpregressor__hidden_layer_sizes': (50,), 'mlpregressor__learning_rate_init': 0.01, 'mlpregressor__max_iter': 1000}
C:\Users\Johns\AppData\Roaming\Python\Python313\site-packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```

MLP (искусственная нейронная сеть) представляет собой полностью связную сеть с несколькими скрытыми слоями. Для регрессии используется для аппроксимации нелинейных зависимостей.

Для обучения модели MLPRegressor из библиотеки scikit-learn был выполнен поиск гиперпараметров по сетке (GridSearchCV). Лучшие параметры:

Архитектура: (50,) (один скрытый слой с 50 нейронами).

Функция активации: relu (ReLU).

Коэффициент регуляризации (alpha): 0.0001 (L2-регуляризация для предотвращения переобучения).

Скорость обучения: 0.01.

Результаты обучения:

MAE на обучающей выборке: 4037.69

MAE на тестовой выборке: 3566.28

Анализ: MLP показал результат MAE=3566.28 на тесте, что близко к результатам градиентного бустинга (CatBoost - 3584). Несмотря на то, что нейронные сети обычно требуют больших объемов данных, MLP удалось достичь приемлемой точности. Стоит отметить, что при обучении было зафиксировано предупреждение о сходимости (ConvergenceWarning), что указывает на то, что увеличение числа итераций (max_iter) могло бы улучшить результат. Однако, при малом количестве данных это также могло бы привести к переобучению.

4.8.7. Метод группового учета аргументов (GMDH)

3.3.12. GMDH (метод группового учета аргументов) ¶

```
print("\n--- Обучение GMDH (МГУА) ---")

if GMDH_AVAILABLE:
    try:
        # Используем комбинаторный алгоритм
        model_gmdh = GMDH(
            criterion='test',
            max_layer=4,
            combination='combi', # Линейный метод COMBI
            normalize=True
        )

        start_time_gmdh = timeit.default_timer()
        model_gmdh.fit(features_train.values, target_train.values)
        training_time_gmdh = timeit.default_timer() - start_time_gmdh

        start_pred_gmdh = timeit.default_timer()
        predictions_gmdh = model_gmdh.predict(features_test.values)
        pred_time_gmdh = timeit.default_timer() - start_pred_gmdh

        result_MAE_v_gmdh = mean_absolute_error(target_test, predictions_gmdh)
        print(f'MAE TEST: {result_MAE_v_gmdh:.4f}')

        results.loc[len(results)] = {
            'NAME': 'GMDH (COMBI)',
            'MAE TRAIN': None,
            'MAE TEST': result_MAE_v_gmdh,
            'PREDICTIONS': predictions_gmdh.mean(),
            'TIME TRAINING [s]': training_time_gmdh,
            'TIME PREDICTION [s]': pred_time_gmdh,
            'PARAMETRS': f'Слоев: {model_gmdh.layer}'
        }
    except Exception as e:
        print(f"Ошибка: {e}. Возможно, нужно установить библиотеку: !pip install gmdhpy")
else:
    print("Библиотека gmdhpy не найдена. Пропускаем обучение GMDH.")

--- Обучение GMDH (МГУА) ---
Библиотека gmdhpy не найдена. Пропускаем обучение GMDH.
```

МГУА — это метод самоорганизации, который строит модель в виде полинома Колмогорова-Габбора, последовательно усложняя структуру на основе внешних критериев. В отличие от обычной регрессии, МГУА сам определяет наиболее значимые переменные и их комбинации.

В данном исследовании была предпринята попытка использования

библиотеки `gmdhpy`.

Результаты:

При попытке импорта библиотеки (`from gmdh import GMDH`) возникла ошибка `ModuleNotFoundError`.

Вывод: Библиотека `gmdhpy` не была установлена в окружении, использованном для выполнения работы.

Решение: Была произведена попытка установки через `!pip install gmdhpy`, однако установка не завершилась успешно, либо библиотека оказалась несовместимой с текущей версией Python/зависимостей. В связи с этим модель МГУА обучить не удалось.

4.9.Итог

После добавления в анализ моделей стекинга и многослойного персептрона, итоговое сравнение показало следующую картину (таблица отсортирована по рейтингу):

Модель	MAE TEST	TIME TRAINING [s]	RATING
LGBMRegressor	3981.63	0.01	0.326
StackingRegressor	3390.83	0.03	0.281
CatBoostRegressor	3584.13	0.05	0.245
BaggingRegressor	3178.17	0.06	0.220
MLPRegressor	3566.28	N/A (0.1+)	N/A
RandomForestRegressor	3220.82	0.10	0.096
AdaBoostRegressor	3479.22	0.09	0.098

(Примечание: Рейтинг может смещаться в зависимости от приоритета метрик (MAE vs Время))

Вывод по моделям:

Лучшая	модель	по	точности
(MAE):	BaggingRegressor	продемонстрировал	наилучшую точность на тестовых данных (3178.17), несмотря на то, что показал не самые низкие значения на обучающей выборке. Это говорит о его хорошей обобщающей способности и устойчивости к переобучению.

Лучшая модель по скорости обучения: LGBMRegressor был обучен практически мгновенно (0.01 сек), но его точность оказалась самой низкой среди всех рассмотренных моделей (3981.63). Это делает его хорошим кандидатом для быстрого прототипирования, но не для финального решения.

Самые	сбалансированные
модели: StackingRegressor и CatBoostRegressor	показали средние результаты как по точности (3390 и 3584), так и по времени обучения, обеспечивая хороший баланс между качеством и скоростью.

Нейронная сеть (MLP): MLPRegressor показал результат (3566.28), который является посредственным. Без тщательного подбора архитектуры и большего объема данных (или использования GPU) нейронные сети уступают классическим ансамблевым моделям в задачах с малым размером выборки.

МГУА: В силу технических причин (отсутствие рабочей библиотеки) обучить и оценить данную модель не представилось возможным. Метод требует дальнейшего исследования.

5. Вывод

В ходе лабораторной работы изучили ансамбли моделей машинного обучения.